

# Prova Finale Reti Logiche A.A. 2024/25

Matteo Aldrigo

(Codice Persona 10844970 - Matricola 214164)

Prof. William Fornaciari - Prof. Federico Reghenzani

Aprile 2025

## Contents

|          |                                                          |           |
|----------|----------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduzione</b>                                      | <b>2</b>  |
| <b>2</b> | <b>Analisi della specifica</b>                           | <b>2</b>  |
| 2.1      | Interfaccia del modulo . . . . .                         | 2         |
| 2.2      | Memoria . . . . .                                        | 3         |
| 2.3      | Filtro e normalizzazione . . . . .                       | 4         |
| 2.4      | Esempio di applicazione del filtro di ordine 3 . . . . . | 5         |
| 2.5      | Esempio di applicazione del filtro di ordine 5 . . . . . | 5         |
| <b>3</b> | <b>Architettura</b>                                      | <b>6</b>  |
| 3.1      | Datapath . . . . .                                       | 6         |
| 3.2      | Macchine a stati finiti (FSM) . . . . .                  | 16        |
| 3.2.1    | Control FSM . . . . .                                    | 16        |
| 3.2.2    | Compute FSM . . . . .                                    | 17        |
| <b>4</b> | <b>Risultati sperimentali</b>                            | <b>21</b> |
| 4.1      | Sintesi . . . . .                                        | 21        |
| 4.2      | Simulazioni . . . . .                                    | 22        |
| <b>5</b> | <b>Conclusioni</b>                                       | <b>24</b> |

# 1 Introduzione

La **Prova Finale di Reti Logiche** ha come obiettivo lo sviluppo di un modulo hardware, descritto in **VHDL**, capace di interfacciarsi con una memoria.

Il modulo, una volta configurato, legge una sequenza di parole rappresentate in complemento a 2 su 1 byte a cui viene applicato un filtro differenziale e scrive la sequenza risultante in memoria.

## 2 Analisi della specifica

### 2.1 Interfaccia del modulo

Il componente da realizzare possiede la seguente interfaccia:

```
entity project_reti_logiche is
  port (
    i_clk : in std_logic;
    i_rst : in std_logic;
    i_start : in std_logic;
    i_add : in std_logic_vector(15 downto 0);
    o_done : out std_logic;
    o_mem_addr : out std_logic_vector(15 downto 0);
    i_mem_data : in std_logic_vector(7 downto 0);
    o_mem_data : out std_logic_vector(7 downto 0);
    o_mem_we : out std_logic;
    o_mem_en : out std_logic
  );
end project_reti_logiche;
```

In particolare:

- **i\_clk**: segnale di *CLOCK*, generato dal Test Bench.
- **i\_rst**: segnale di *RESET* asincrono, che inizializza il modulo.
- **i\_start**: segnale di *START*, generato dal Test Bench, che permette di avviare una nuova computazione.
- **i\_add**: segnale (vettore) che indica l'indirizzo di partenza dei dati in memoria.

- **o\_done**: segnale che comunica la fine dell'elaborazione.
- **o\_mem\_addr**: segnale (vettore) che indica l'indirizzo utilizzato durante le letture e scritture in memoria.
- **i\_mem\_data**: segnale (vettore) che contiene il dato richiesto in seguito ad una lettura.
- **o\_mem\_data**: segnale (vettore) che contiene il dato che deve essere scritto in memoria.
- **o\_mem\_en**: segnale che permette di abilitare la memoria.
- **o\_mem\_we**: segnale che indica il tipo di operazione richiesta alla memoria: 0 per la lettura, 1 per la scrittura.

## 2.2 Memoria

La memoria con cui si interfaccia il modulo utilizza un indirizzamento al byte ed è di tipo sincrono, ovvero tutte le operazioni di lettura e scrittura sono sincronizzate con il segnale di clock.

I dati necessari per la configurazione del modulo sono memorizzati a partire dall'indirizzo specificato sull'ingresso **i\_add**. Questi dati includono:

- **K1 e K2**: due byte che, combinati, definiscono la lunghezza della sequenza da elaborare ( $K = K1 \mid K2$ ), dove  $K1$  è il byte più significativo.
- **S**: un byte, di cui è rilevante solo il bit meno significativo. Se tale bit è 0, verrà applicato il filtro di ordine 3, altrimenti quello di ordine 5.
- **$C_1, \dots, C_{14}$** : quattordici byte contenenti i coefficienti per i due filtri. In particolare, i primi sette ( $C_1, \dots, C_7$ ) sono relativi al filtro di ordine 3, mentre gli ultimi sette ( $C_8, \dots, C_{14}$ ) a quello di ordine 5.

Sia i dati da processare che i dati elaborati sono parole da un byte in complemento a due, rappresentanti valori compresi tra  $-128$  e  $+127$ . La sequenza di ingresso, lunga  $K$  byte, è disponibile a partire dall'indirizzo  $i\_add + K$ , mentre la sequenza risultante è scritta a partire dall'indirizzo  $i\_add + 17 + K$ .

| Memoria          |           |                                         |
|------------------|-----------|-----------------------------------------|
| Indirizzo        | Contenuto | Descrizione                             |
| i_add            | $K1$      | Byte più significativo di $K$           |
| i_add + 1        | $K2$      | Byte meno significativo di $K$          |
| i_add + 2        | $S$       | Determina il filtro da utilizzare       |
| i_add + 3        | $C_1$     | Primo coefficiente del filtro           |
| $\vdots$         | $\vdots$  | $\vdots$                                |
| i_add + 16       | $C_{14}$  | Ultimo coefficiente del filtro          |
| i_add + 17       | $W_1$     | Primo dato della sequenza da processare |
| $\vdots$         | $\vdots$  | $\vdots$                                |
| i_add + 17 + $K$ | $R_1$     | Primo dato della sequenza elaborata     |
| $\vdots$         | $\vdots$  | $\vdots$                                |

Table 1: Configurazione della memoria

### 2.3 Filtro e normalizzazione

Il filtro da applicare alla sequenza di parole lette ha la seguente funzione:

$$f'(i) = \frac{1}{n} \sum_j C_j \cdot f[j + i]$$

dove il simbolo  $j$  rappresenta un indice nell'intervallo  $[-2, 2]$  per un filtro di ordine 3 e  $[-3, 3]$  per un filtro di ordine 5. Il coefficiente di normalizzazione  $n$  assume il valore di 12 per il filtro di ordine 3 e 60 per quello di ordine 5.

La divisione dovuta alla normalizzazione deve essere approssimata nel seguente modo:

- $\frac{1}{12} \approx \frac{1}{16} + \frac{1}{64} + \frac{1}{256} + \frac{1}{1024}$
- $\frac{1}{60} \approx \frac{1}{64} + \frac{1}{1024}$

**Osservazione:** Uno shift a destra di  $m$  bit equivale a una divisione per  $2^m$  con troncamento. Per i numeri positivi il risultato viene arrotondato verso 0, mentre per i numeri negativi viene arrotondato verso  $-1$ , introducendo un errore. Per correggere questo effetto, si aggiunge  $+1$  nel caso di shift di numeri negativi, mentre per i numeri positivi il risultato rimane invariato.

## 2.4 Esempio di applicazione del filtro di ordine 3

Si consideri un filtro di ordine 3 applicato a una sequenza di 15 parole.  
I coefficienti presenti in memoria sono:

$$[0, -1, 8, 0, -8, 1, 0, 1, -9, 45, 0, -45, 9, -1]$$

La sequenza di parole è:

$$[105, 45, -69, 84, -111, 60, -118, -72, -122, -52, 100, 13, 47, -59, -108]$$

L'applicazione del filtro fornisce la seguente sequenza di output:

$$[-33, 121, -42, 27, 10, -7, 86, -5, 3, -128, -27, 33, 29, 100, -42]$$

## 2.5 Esempio di applicazione del filtro di ordine 5

Si consideri un filtro di ordine 5 applicato a una sequenza di 24 parole.  
I coefficienti presenti in memoria sono:

$$[0, -1, 8, 0, -8, 1, 0, 1, -9, 45, 0, -45, 9, -1]$$

La sequenza di parole in ingresso è:

$$[32, -24, -35, 0, 46, -54, -39, -22, -53, -47, 12, 11, 11, 45, -30, -14, -35, -25, -19, -35, -41, -61, -24, -62]$$

L'applicazione del filtro fornisce la seguente sequenza di output:

$$[11, 48, -14, -63, 39, 59, -37, 11, 24, -43, -34, 13, -31, 27, 37, -5, 10, -14, 6, 10, 19, -17, 5, -8]$$

### Osservazioni

- Tutti i valori (coefficienti, input e output) sono rappresentati in complemento a due su 8 bit, quindi appartengono all'intervallo  $[-128, 127]$ .
- Il filtro di ordine 3 sfrutta i primi sette coefficienti  $C_1, \dots, C_7 = [0, -1, 8, 0, -8, 1, 0]$ , mentre quello di ordine 5 sfrutta i restanti sette  $C_8, \dots, C_{14} = [1, -9, 45, 0, -45, 9, -1]$ .

## 3 Architettura

### 3.1 Datapath

Il Datapath è composto da tre moduli principali. Il primo si occupa di memorizzare e mantenere nei registri i valori necessari alla computazione, come l'indirizzo di partenza dei dati, la lunghezza della sequenza, l'ordine del filtro e calcola gli indirizzi di memoria per le letture e scritture. Il secondo modulo gestisce i contatori, utili per determinare lo stato corrente del sistema e verificare se la sequenza di parole è stata completamente letta e la computazione conclusa. Infine, il terzo modulo gestisce le operazioni per calcolare i valori da memorizzare in memoria. Sebbene nelle figure non sempre appaiano, i segnali di clock e reset sono correttamente implementati nell'architettura reale.

#### Memorizzazione e calcolo indirizzi

Questo modulo si occupa sia di memorizzare e mantenere nel tempo i valori necessari alla computazione, ma anche di calcolare gli indirizzi di memoria per le operazioni di lettura e scrittura. Esso è composto da due moduli indipendenti:

1. **Indirizzo iniziale e lunghezza della sequenza:** Nell'immagine 1 si possono notare i registri che memorizzano i valori di *initial\_addr\_reg* e di *k\_reg*. Entrambi i multiplexer hanno come ingresso il segnale *data*, che rappresenta il primo segnale valido a cui viene settato il registro e successivamente il valore presente nel registro stesso che deve essere mantenuto nel tempo.

Si noti inoltre che, per brevità e per garantire coerenza con il codice VHDL, sono stati omessi i segnali di *load*, poiché sarà compito della Compute FSM (3.2.2) determinare quando settare il valore in tali registri.

Infine, questo modulo è in grado di calcolare l'indirizzo di memoria per le operazioni di lettura e scrittura grazie ad un sommatore, il quale utilizza come ingressi i valori presenti nei registri e un valore dinamico fornito tramite il segnale *add\_value*.

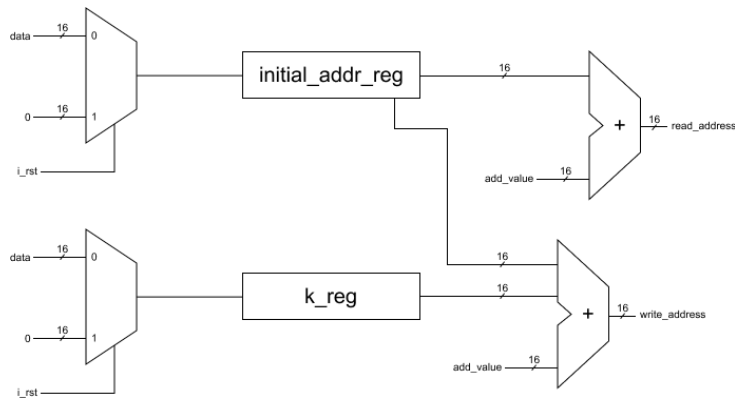


Figure 1: Indirizzi di memoria

2. **Ordine del filtro:** Questa parte del modulo (rappresentata nella figura 2) si occupa di memorizzare e mantenere un valore nel tempo, necessario per determinare l'ordine del filtro da utilizzare. Il ragionamento svolto sul segnale *data* è analogo al caso precedente.

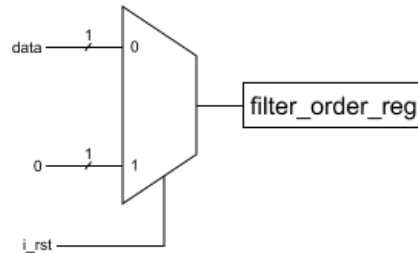


Figure 2: Ordine del filtro

## Gestione contatori

Questo modulo gestisce quattro contatori indipendenti:

1. **isRamReady counter:** Come illustrato nell'immagine 3, il contatore isRamReady inizia con valore 0. Se necessario, viene incrementato di 1 fino a raggiungere il valore 2. Dunque, i valori assunti dal contatore seguono il ciclo:  $0 \rightarrow 1 \rightarrow 2 \rightarrow 0 \rightarrow \dots$

Questo contatore è essenziale per gestire correttamente le richieste alla memoria, poiché, essendo sincrona, il dato sarà disponibile al ciclo di

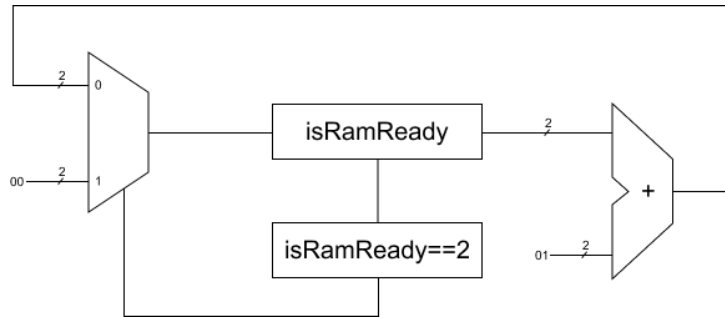


Figure 3: isRamReady counter

clock successivo. Per garantire la stabilizzazione del dato richiesto, si attende un ulteriore ciclo di clock prima di accedervi. Pertanto, nell'architettura, quando isRamReady è pari a 0 la richiesta di lettura viene inviata alla memoria, mentre quando è pari a 2 viene letto e utilizzato il valore ritornato da quest'ultima.

2. **Coefficient counter:** Come mostrato nella figura 4, il contatore viene inizializzato a 0. Quando necessario viene incrementato di 1 fino a raggiungere il valore 6, momento in cui viene automaticamente reinizializzato.

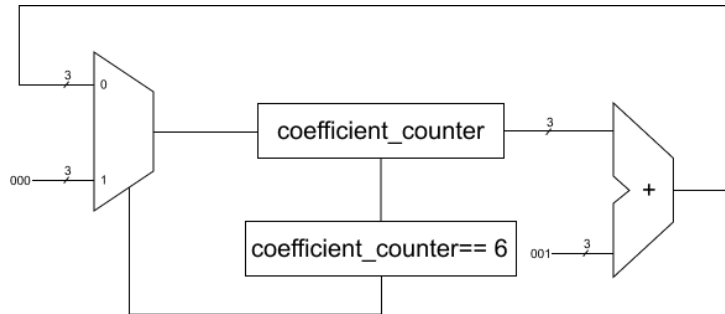


Figure 4: Coefficient counter

Il contatore viene utilizzato per selezionare il registro corretto in cui memorizzare i coefficienti provenienti dalla memoria. Esso è stato impiegato per semplificare l'accesso all'array di registri utilizzato in VHDL, ma anche per effettuare la transizione di stato una volta caricati tutti i coefficienti.



3. **Computation step counter** : Come illustrato nella figura 5, il contatore parte dal valore 0. Viene incrementato, quando necessario, di 1 fino a raggiungere il valore 2, istante in cui viene reinizializzato.

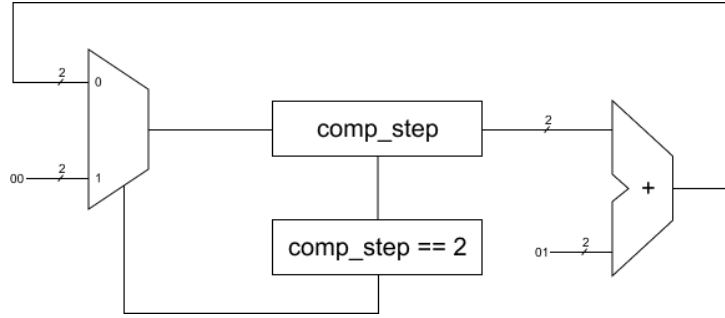


Figure 5: Computation step counter

Il contatore è utilizzato per permettere alla Compute FSM (3.2.2) di identificare lo stato corrente della computazione della parola. Le fasi che individua sono tre: somma dei prodotti tra coefficienti e parole, normalizzazione del risultato ed infine verifica della necessità di saturazione con estrazione degli 8 bit meno significativi.

4. **Words counter**: Il contatore mostrato nella figura 6 inizialmente assume il valore 0 e, quando necessario, viene incrementato di 1 fino a raggiungere  $2^{17}$ , momento in cui viene reinizializzato automaticamente.

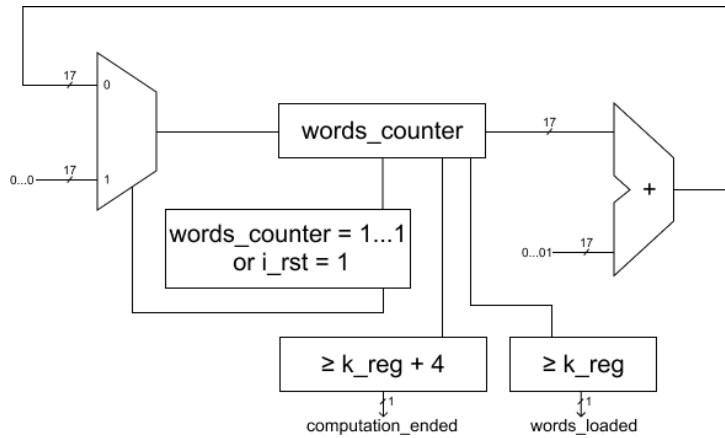


Figure 6: Words counter

Questo contatore consente di stabilire sia il completamento della lettura dell'intera sequenza di parole (quando il suo valore è  $\geq k\_reg$ ), sia il termine della computazione della stessa (quando il suo valore è  $\geq k\_reg + 4$ ).

Si osservi che il registro del contatore deve essere di 17 bit, poiché la lunghezza della sequenza di parole è rappresentata da un valore di 16 bit. Se così non fosse, il confronto con  $k\_reg + 4$  risulterebbe impossibile nel caso limite  $k\_reg = 2^{16}$ , a causa dell'overflow.

**Osservazione:** I contatori *Coefficient Counter* e *Words Counter* vengono utilizzati anche per calcolare alcuni indirizzi di memoria necessari durante la computazione.

## Computazione delle parole

Il datapath, illustrato in Figura 11, elabora la sequenza di parole in ingresso, producendo per ognuna di esse il valore *o\_data*. Per semplificarne la comprensione e seguire il processo logico con cui è stato realizzato, l'intera struttura è stata scomposta nei moduli elementari, ciascuno dei quali svolge una funzionalità specifica.

1. **Coefficient register:** Il registro dei coefficienti, mostrato in figura 7, è composto da sette registri, ciascuno associato ad un coefficiente. L'attivazione di ciascun registro avviene quando il segnale *coefficient\_counter* assume il valore corrispondente (ad esempio, il registro 1 si attiva quando *coefficient\_counter* = 1, il registro 2 quando *coefficient\_counter* = 2, e così via). Per supportare entrambe le configurazioni del filtro, di ordine 3 e 5, è stato impiegato un multiplexer che, nel caso di filtro di ordine 3, forza a zero i coefficienti 1 e 7, poiché non utilizzati nella computazione; mentre, nel caso del filtro di ordine 5 l'ingresso per tali registri è pari a *i\_mem\_data*, ovvero il valore letto dalla memoria.
2. **Words shift register:** Lo shift register, illustrato in figura 8, è progettato per memorizzare le parole necessarie alla computazione. Ad ogni ciclo di caricamento, il contenuto dei registri viene shiftato di una posizione: il valore presente nel registro *words\_1* viene trasferito nel registro *words\_2*, e così via. Inizialmente, tutti i registri sono inizializzati a zero, pertanto, durante la fase di configurazione del

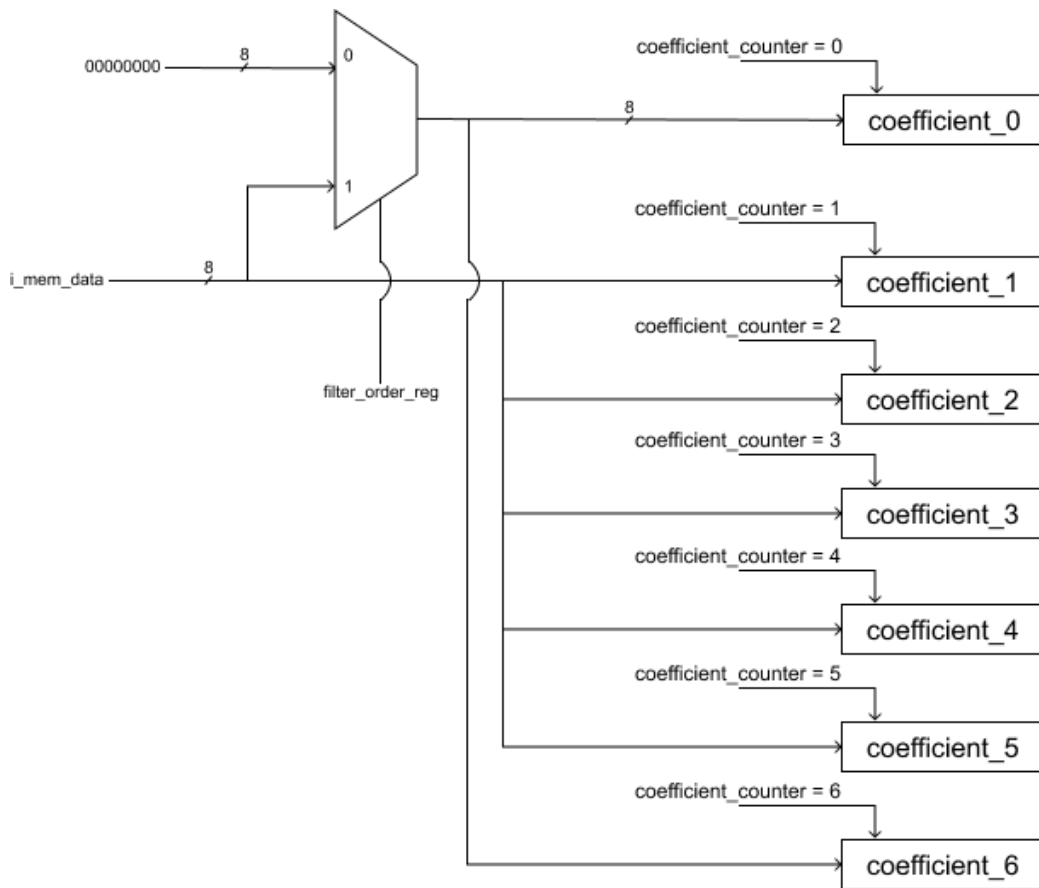


Figure 7: Registri dei coefficienti

modulo, il caricamento delle prime quattro parole consente di avviare la computazione, considerando come zero i valori precedenti l'inizio della sequenza. Dunque, il valore in uscita dal multiplexer corrisponde a *i\_mem\_data* finché la sequenza non viene completamente letta, assumendo poi il valore zero per gestire i dati inesistenti successivi alla fine della stessa. Si sottolinea che, nel codice VHDL, non è previsto un segnale esplicito per effettuare lo shift, in quanto tale operazione è gestita dalla FSM che controlla le transizioni tra i registri.

3. **Applicazione del filtro:** Per l'applicazione del filtro sulle parole si utilizzano i due sotto-moduli analizzati in seguito.

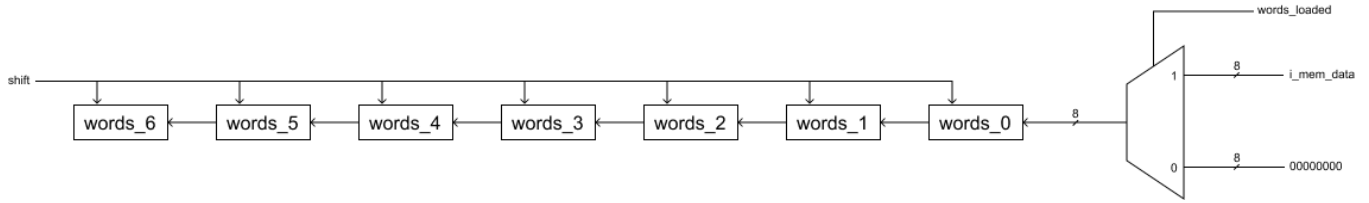


Figure 8: Words shift register

- **Somma dei prodotti:** Il modulo illustrato in figura 9 è responsabile della realizzazione dei prodotti tra i coefficienti e i valori memorizzati nei registri, operazione che viene eseguita quando il segnale *computation\_step* è pari a 0. In particolare, per ciascuna coppia coefficiente-registro, viene calcolato il prodotto che sarà poi sommato all'accumulatore *products\_sum*. Formalmente, l'operazione svolta può essere espressa mediante la seguente equazione:

$$products\_sum = \sum_{k=0}^6 coefficients(k) \cdot words(6 - k)$$

Questa operazione consente di preparare il risultato parziale della computazione per la successiva fase di normalizzazione.

**Nota:** Il risultato è codificato su 18 bit. Tale dimensione è necessaria in quanto vengono sommati 7 prodotti di coppie di numeri a 8 bit in complemento a 2.

- **Normalizzazione:** Il modulo responsabile della normalizzazione, illustrato in Figura 10, opera in due fasi differenti in base al valore del segnale *computation\_step*. Quando *computation\_step* = 1, il modulo esegue la normalizzazione del valore prodotto nello step precedente, distinguendo tra i casi di filtro di ordine 3 e di ordine 5. Si noti che, ad ogni computazione, vengono calcolati i valori per entrambi i filtri; successivamente, tramite un multiplexer, viene selezionato quello relativo al filtro attualmente in uso (per ulteriori dettagli sul processo di normalizzazione, si rimanda alla Sezione 2.3). Al termine di questa fase il registro *normalized\_sum* contiene il valore normalizzato. Quando invece *computation\_step* = 2 il

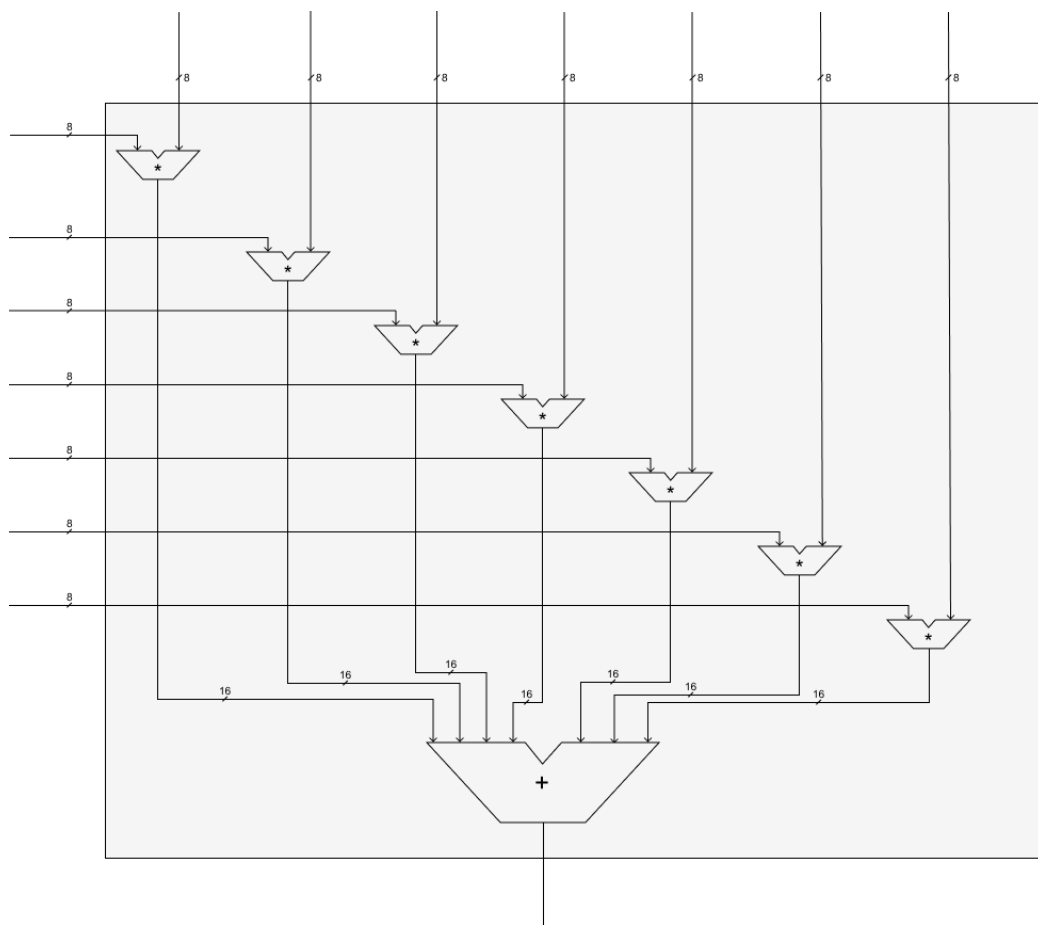


Figure 9: Blocco sommatore dei 7 prodotti

modulo passa alla fase di saturazione del valore normalizzato. In questa fase vengono utilizzati solo gli 8 bit meno significativi del registro *normalized\_sum*, poiché il risultato finale deve essere di un singolo byte. Un multiplexer dotato di due selettori verifica se il valore normalizzato supera determinati limiti: se è maggiore o uguale a 127, viene saturato a 127; se è minore o uguale a -128, viene saturato a -128; mentre per valori compresi tra questi limiti il segnale viene lasciato invariato. Tale comportamento può essere formalizzato con la seguente espressione:

$$o\_data = \begin{cases} 127 & \text{se } normalized\_sum \geq 127, \\ normalized\_sum_{[7:0]} & \text{se } -128 < normalized\_sum < 127, \\ -128 & \text{se } normalized\_sum \leq -128. \end{cases} \quad (1)$$

Al termine della fase di saturazione il segnale  $o\_data$  contiene il valore finale dell'elaborazione, pronto per essere salvato in memoria.

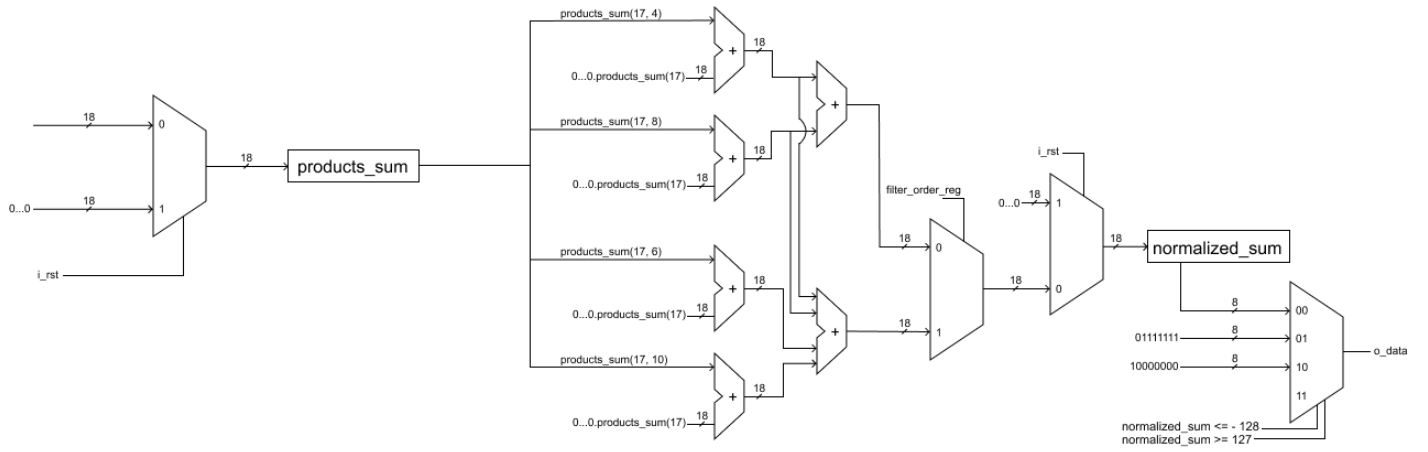


Figure 10: Normalizzatore

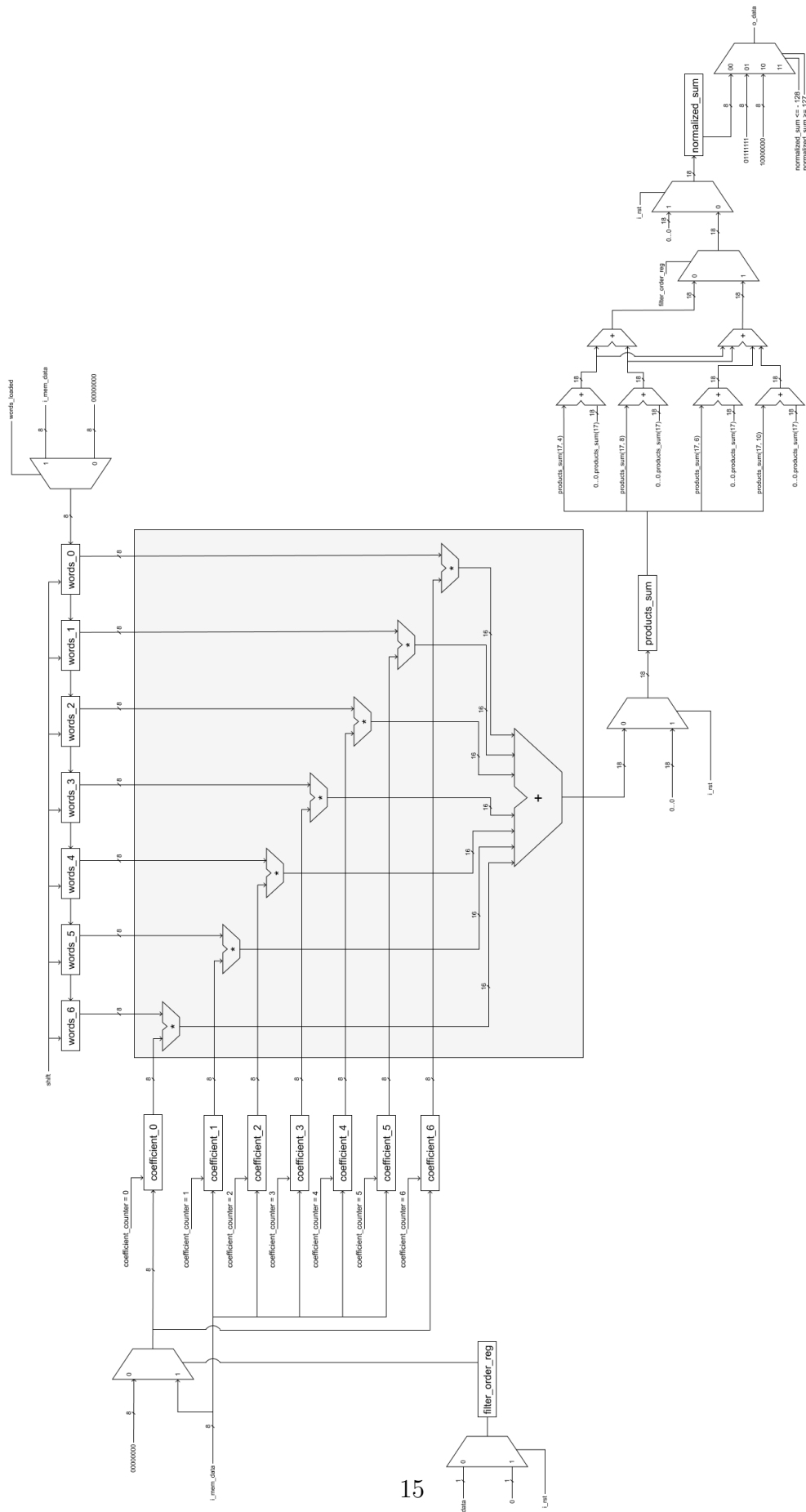


Figure 11: Datapath

## 3.2 Macchine a stati finiti (FSM)

Il modulo realizzato utilizza due macchine a stati finiti (FSM). La prima regola gli stati operativi in cui si può trovare il componente durante l'esecuzione, mentre la seconda gestisce l'intero processo di computazione. Quest'ultima entra in funzione solo quando la prima FSM si trova nello stato *execute*.

### 3.2.1 Control FSM

La prima macchina a stati finiti, denominata Control FSM, gestisce gli stati operativi del componente. Il suo compito principale è abilitare la seconda FSM (3.2.2) e segnalare all'interfaccia il termine dell'elaborazione. Nei paragrafi successivi verranno analizzati in dettaglio i singoli stati e le relative transizioni.

**Osservazioni sul diagramma degli stati** Per rendere il diagramma degli stati delle due FSM più chiaro e conciso, non sono state riportate le transizioni ridondanti che, in caso di reset asincrono, riportano ogni stato al reset. Sebbene non esplicitate nel diagramma, tali transizioni sono state comunque considerate e implementate correttamente nel componente.

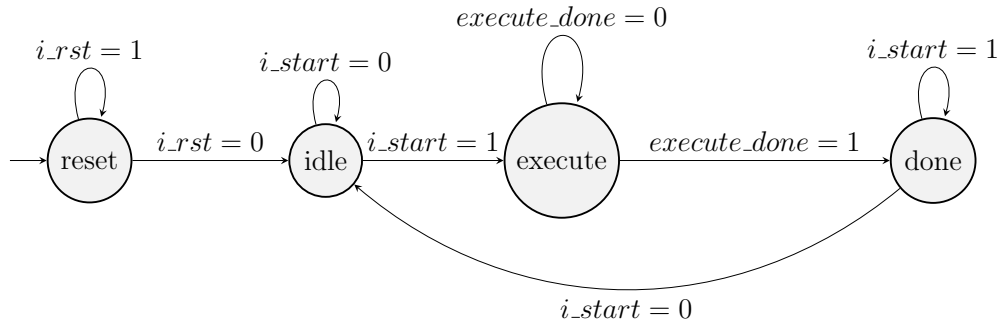


Figure 12: Control FSM: diagramma degli stati

### Analisi degli stati

- **Reset:** Stato iniziale in cui il componente si trova subito dopo l'attivazione del segnale di reset ( $i\_rst = 1$ ). In questo stato il segnale di uscita *o\_done* viene impostato a 0. La transizione allo stato *Idle*



avviene quando il segnale  $i\_rst$  torna a 0. Inoltre, per garantire il reset asincrono, il sistema torna in questo stato ogni volta che viene ricevuto  $i\_rst = 1$

- **Idle:** Stato di attesa in cui il componente attende l'arrivo di una nuova elaborazione. Rimane in questo stato finché il segnale di avvio ( $i\_start$ ) è 0. Quando  $i\_start = 1$ , la FSM passa allo stato *Execute*.
- **Execute:** stato in cui viene attivata la seconda FSM (3.2.2). Durante questa fase,  $o\_done$  rimane a 0. La FSM rimane in questo stato finché il segnale  $execute\_done$ , prodotto dalla seconda FSM, non diventa 1, segnalando la fine dell'elaborazione.
- **Done:** stato finale in cui la computazione è terminata e il segnale  $o\_done$  viene impostato a 1 per notificare il completamento dell'operazione. Se  $i\_start = 0$ , la FSM ritorna in *Idle* in attesa di un nuovo comando.

## Osservazioni

Il modulo è progettato per supportare elaborazioni successive in modo continuo. È importante notare che solo durante la prima computazione viene garantita l'inizializzazione del modulo. Nelle elaborazioni successive, invece, a meno che non venga fornito esplicitamente un segnale di reset, il modulo deve essere in grado di funzionare correttamente senza necessitare di un reset tra una computazione e l'altra.

### 3.2.2 Compute FSM

La seconda macchina a stati finiti, detta Compute FSM, gestisce gli stati relativi all'elaborazione della sequenza di parole in ingresso. Essa si occupa di inizializzare il componente con i parametri di configurazione, per poi procedere all'elaborazione della sequenza e alla conseguente scrittura dei risultati in memoria.

**Osservazione:** alcuni stati presentano denominazioni leggermente diverse rispetto a quelle utilizzate nel codice VHDL. Questa scelta è stata adottata per facilitare la rappresentazione grafica degli stati, evitando nomi eccessivamente lunghi che avrebbero compromesso la chiarezza dello schema.

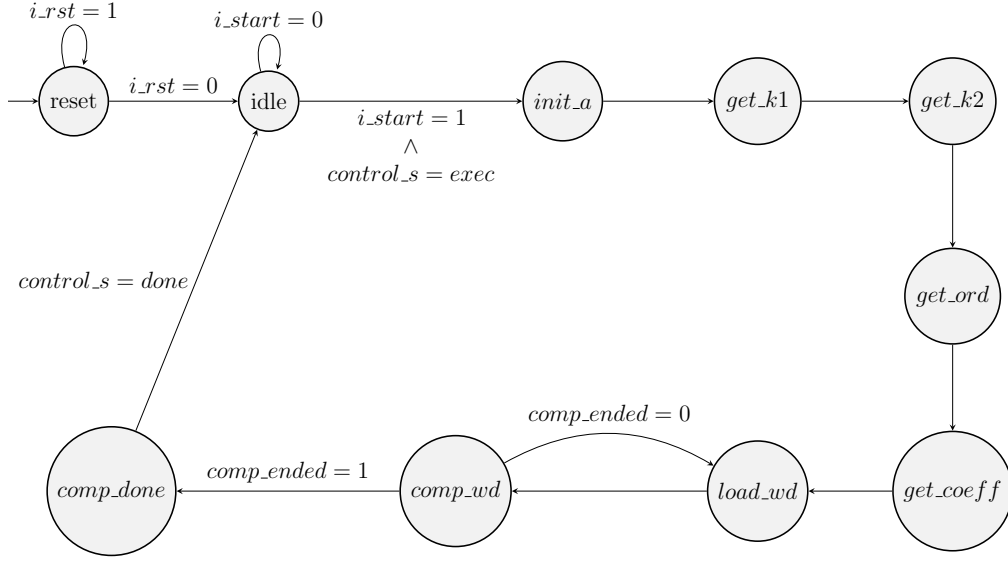


Figure 13: Compute FSM: diagramma degli stati

**Analisi degli stati** Per ottimizzare il numero di stati nella FSM, è stato introdotto un contatore, denominato *isRamReady*, che consente di richiedere un dato alla memoria, attendere la sua stabilizzazione e utilizzarlo all'interno dello stesso stato. In questo modo, si evita l'introduzione di numerosi stati di attesa ogni volta che è necessario accedere alla memoria, centralizzando la gestione di queste operazioni all'interno degli stati interessati. Grazie a questo approccio, si è passati da una prima versione della FSM con oltre 40 stati a una versione molto più compatta che ne utilizza solamente 10.

- **Reset:** Stato iniziale in cui il componente si trova subito dopo l'attivazione del segnale di reset ( $i\_rst = 1$ ). In questo stato vengono inizializzati tutti i segnali e i registri utilizzati durante l'elaborazione. La transizione allo stato *Idle* avviene quando il segnale  $i\_rst$  torna a 0. Inoltre, per garantire il reset asincrono, il sistema torna in questo stato ogni volta che viene ricevuto  $i\_rst = 1$ .
- **Idle:** Stato in cui il componente attende l'arrivo di una nuova elaborazione finché il segnale di start è pari a 0, oppure se la Control FSM (3.2.1) si trova in uno stato differente da quello di *execute*, altrimenti passa a *Init\_a*. In questo stato, i registri utilizzati

durante la computazione vengono reinizializzati per consentire eventuali elaborazioni successive, senza la necessità di un reset esplicito.

- ***Init\_a***: Stato in cui viene settato il registro *initial\_addr\_reg* al valore presente sull'ingresso *i\_add*.
- ***Get\_k1***: Stato che si occupa di caricare gli 8 bit più significativi nel registro *k\_reg*, in particolare se:
  - *isRamReady* = 0, richiede il dato presente all'indirizzo *i\_add* alla memoria
  - *isRamReady* = 2, scrive i bit più significativi nel registro
- ***Get\_k2***: Stato che si occupa di caricare gli 8 bit meno significativi nel registro *k\_reg*, in particolare se:
  - *isRamReady* = 0, richiede il dato presente all'indirizzo *i\_add* + 1 alla memoria
  - *isRamReady* = 2, scrive i bit meno significativi nel registro
- ***Get\_ord***: Stato che si occupa di caricare il bit meno significativo del parametro *S* nel registro *filter\_order\_reg*, in particolare se:
  - *isRamReady* = 0, richiede il dato presente all'indirizzo *i\_add* + 2 alla memoria
  - *isRamReady* = 2, scrive il bit meno significativo nel registro
- ***Get\_coef***: Stato che carica i 7 coefficienti utilizzando il *coefficient\_counter*. Una volta completato il caricamento, la FSM transita allo stato *Load\_words*. Grazie all'utilizzo del contatore, è stato possibile ridurre il numero di stati utilizzati a uno solo. Si noti che, nel caso di filtro di ordine 3 i registri relativi a *c1* e *c7* sono impostati a 0. In particolare quando:
  - *isRamReady* = 0: richiede il dato per il prossimo coefficiente. L'indirizzo di memoria dipende dal filtro e dal coefficiente in corso:
    - \* Per il filtro di ordine 3, l'indirizzo per l'*n*-esimo coefficiente è  $i\_add + 3 + n$ , dove *n* varia da 0 a 6.

- \* Per il filtro di ordine 5, l'indirizzo per l' $n$ -esimo coefficiente è  $i\_add + 3 + 7 + n$ , dove  $n$  varia da 0 a 6.
- $isRamReady = 2$ : salva l' $n$ -esimo coefficiente nel relativo registro.
- **Load\_wd**: Questo stato si occupa di caricare le parole necessarie alla computazione nello shift register. Grazie all'impiego di un contatore che tiene traccia del numero di parole lette dalla sequenza, il sistema distingue due fasi operative: la configurazione iniziale (caricamento delle prime quattro parole) e il normale flusso di caricamento. Durante la fase di configurazione lo stato rimane attivo fino al completo caricamento delle prime quattro parole; una volta terminato, si passa automaticamente allo stato di computazione. Nei cicli successivi, invece, viene caricata una sola parola per poi procedere con la computazione. Questa soluzione evita l'impiego di quattro stati distinti per la sola configurazione, permettendo l'utilizzo di un unico stato che gestisce entrambe le casistiche. Per ciascuna parola caricata, il processo si articola nelle seguenti fasi:
  - $isRamReady = 0$ : viene richiesta alla memoria la prossima parola necessaria alla computazione. L'indirizzo della parola da leggere è calcolato come:
 
$$o\_mem\_addr = initial\_addr\_reg + 17 + words\_counter$$
  - $isRamReady = 2$ : la nuova parola viene inserita nel registro dedicato.
- **Comp\_wd**: Questo stato gestisce tutte le fasi necessarie per computare le parole. In particolare quando:
  - $computation\_step = 0$ : viene applicato il filtro che esegue la somma dei prodotti tra le coppie di registri coefficienti-parole ( si veda 3 per ulteriori dettagli).
  - $computation\_step = 1$ : viene effettuata la normalizzazione (si rimanda a 3 per i dettagli).
  - $computation\_step = 2$ : viene valutato se è necessario saturare il valore ottenuto ai limiti dell'intervallo possibile  $[-128, +127]$  (si

veda 3 per eventuali chiarimenti). Inoltre, questa fase si occupa anche di scrivere il dato prodotto in memoria all'indirizzo:

$$o\_mem\_addr = initial\_addr\_reg + 13 + k\_reg + words\_counter$$

Infine, questo stato imposta il segnale *computation\_ended* a 1 quando  $words\_counter \geq k\_reg + 4$ , indicando che la computazione è terminata e consentendo la transizione allo stato di *comp\_done*. Se invece *computation\_ended* risulta essere pari a 0, la computazione prosegue ritornando nello stato *load\_wd*.

- **Comp\_done:** Questo stato serve a notificare alla Control FSM (3.2.1) che la computazione della sequenza è stata completata, tramite l'assegnamento del segnale *execute\_done* a 1. Al ciclo di clock successivo, la Compute FSM effettua la transizione verso lo stato di *idle*, restando in attesa di una nuova elaborazione.

## 4 Risultati sperimentali

### 4.1 Sintesi

Il modulo è stato sintetizzato mediante il software Vivado, utilizzando come FPGA target **Artix-7 FPGA xc7a200tfbg484-1**. Nella seguente tabella sono riportati i componenti della FPGA utilizzati.

| Site Type             | Used | Fixed | Available | Util% |
|-----------------------|------|-------|-----------|-------|
| Slice LUTs*           | 869  | 0     | 134600    | 0.65  |
| LUT as Logic          | 869  | 0     | 134600    | 0.65  |
| LUT as Memory         | 0    | 0     | 46200     | 0.00  |
| Slice Registers       | 240  | 0     | 269200    | 0.09  |
| Register as Flip Flop | 240  | 0     | 269200    | 0.09  |
| Register as Latch     | 0    | 0     | 269200    | 0.00  |
| F7 Muxes              | 0    | 0     | 67300     | 0.00  |
| F8 Muxes              | 0    | 0     | 33650     | 0.00  |

Table 2: Utilizzo delle risorse FPGA

Da questi dati si può notare che non sono stati inferiti latch. L'assenza di latch implica che la descrizione del circuito è stata realizzata in modo

da evitare ambiguità nei processi combinatori, garantendo così una sintesi affidabile e sincrona.

Dall'analisi del report di timing risulta che il ritardo massimo del datapath è pari a 14.817 ns, con uno slack time di 5.065 ns, il che significa che il percorso dati più critico opera con un margine di sicurezza all'interno del periodo di clock massimo di 20 ns. Inoltre, il critical path risulta essere composto da 19 livelli e il ritardo è ben bilanciato, con circa il 52% attribuibile alla logica e il 48% al routing.

Questi risultati evidenziano una progettazione efficace, in cui i parametri chiave rispettano i vincoli temporali imposti, garantendo il corretto funzionamento del circuito.

## 4.2 Simulazioni

Il modulo è stato testato, sia in modalità Behavioral che in Post-Synthesis Simulation. I test effettuati hanno l'obiettivo di verificare sia la correttezza dell'output del componente che la sua robustezza nell'affrontare eventuali corner case.

### Test bench filtro di ordine 3

Il primo test bench utilizzato ci è stato fornito insieme alla specifica del progetto e ha l'obiettivo di verificare un'intera computazione utilizzando il filtro di ordine 3.

### Test bench filtro di ordine 5

Il secondo test bench utilizzato è stato ottenuto modificando quello iniziale. In particolare è stato cambiato a 1 il bit meno significativo del byte che identifica l'ordine del filtro ( $S$ ), in modo tale da selezionare il filtro di ordine 5. Successivamente, sono stati ricalcolati i valori di uscita per garantire la correttezza dell'output del componente. Questo test bench ha quindi lo scopo di testare l'intera computazione tramite un filtro di ordine 5, come mostrato nell'esempio 2.5.

### Test bench reset dopo una computazione

Questo test bench è stato realizzato per verificare che il componente risponda correttamente a un segnale di reset generato tra la fine della computazione

corrente e l'inizio di quella successiva. L'obiettivo è assicurarsi che, al reset, tutti i registri e i segnali interni vengano reinizializzati correttamente, in modo da consentire al componente di ripartire senza errori con una nuova elaborazione.

### **Test bench reset durante la computazione**

Questo test bench è stato progettato per valutare il comportamento del componente quando il segnale di reset asincrono viene attivato durante una computazione. In questo scenario, l'obiettivo è la tempestività e qualità della risposta al reset: la verifica consiste nel controllare che, nonostante l'interruzione della computazione, i registri interni vengano correttamente inizializzati per preparare il modulo a una successiva elaborazione.

### **Test bench computazioni consecutive senza reset**

In questo test bench è stata verificata la capacità del modulo di eseguire elaborazioni consecutive senza la necessità di un reset per reinizializzare i registri. Per valutare ulteriormente la robustezza del sistema, sono state utilizzate due sequenze di ingresso differenti, in modo da dimostrare che il modulo è in grado di aggiornare dinamicamente i propri valori e riconfigurarsi correttamente per ogni nuova computazione.

### **Test bench computazioni consecutive senza reset e con indirizzo di memoria differente**

Analogamente al test bench precedente, in questo scenario sono state eseguite due computazioni consecutive senza l'uso del reset per re-inizializzare i registri. La particolarità di questo test bench consiste nel fatto che le due computazioni sfruttano valori iniziali differenti per i dati in memoria (variando il segnale *i\_add*, verificando così la capacità del modulo di adattarsi a differenti indirizzi di memoria tra diverse computazioni.

### **Test bench filtro di ordine 3 seguito da uno di ordine 5**

In questo test bench si è verificata la capacità di eseguire due elaborazioni consecutive, passando da un filtro di ordine 3 a un filtro di ordine 5. Oltre alla modifica del filtro, è stata variata anche la sequenza di ingresso, al fine di dimostrare che il modulo è in grado di gestire computazioni successive

senza la necessità di un reset. L'obiettivo del test è verificare la capacità del componente di ricaricare dinamicamente i valori e di riconfigurarsi correttamente per ogni nuova elaborazione.

### **Test bench computazione con saturazione dei valori**

In questo test bench è stato analizzato il comportamento del modulo responsabile della saturazione dei valori che eccedono l'intervallo  $[-128, +127]$ . La sequenza in ingresso è stata appositamente progettata per generare output che superino questi limiti. L'obiettivo del test è verificare la corretta applicazione della saturazione, assicurando che i valori in uscita vengano limitati ai valori massimi e minimi consentiti.

## **5 Conclusioni**

Il modulo sintetizzato ha superato con successo tutti i test a cui è stato sottoposto, sia in simulazione Behavioral che in Post-Synthesis Simulation.

Rispetto alla prima versione realizzata, è stato possibile ottimizzare il numero di stati impiegati grazie all'utilizzo di diversi contatori che hanno permesso di gestire più operazioni all'interno dei vari stati. Ad esempio, il caricamento dei coefficienti è stato significativamente semplificato: nella versione precedente, i casi di filtro di ordine 3 e ordine 5 erano gestiti separatamente, mentre nell'attuale implementazione tutti i coefficienti vengono processati all'interno del medesimo stato. Tale approccio ha consentito una riduzione di ben 13 stati.

Anche la configurazione iniziale delle parole è stata semplificata. Infatti, in precedenza erano necessari quattro stati per l'inizializzazione delle parole da processare. Nella versione attuale il processo viene gestito all'interno di un unico stato che, tramite appositi controlli, determina la condizione in cui si trova il componente.

Infine, l'utilizzo del contatore *isRamReady* ha permesso di realizzare un accesso ai dati in memoria in modo comodo e robusto, anche in presenza di ritardi imprevisti, come evidenziato durante le simulazioni Post-Synthesis, in cui il comportamento del sistema si avvicina a quello reale.